

Компьютерный практикум

Тема 4-6. Напоминания: Строки, Массивы, Функции и ещё
немного

Преподаватель: Д. В. Давыдов

2026

Строки: основное

- Строка в Java — неизменяемая последовательность символов.
- Индексация символов начинается с нуля.
- Метод `substring(start, end)` не включает символ с индексом `end`.

```
String s = "Hello";
String s2 = new String(new char[]{'h', 'e', 'l', 'l', 'o'});
String s3 = new String("Hello");
int n = s.length();           // 5
char c = s.charAt(0);         // H
String sub = s.substring(1, 3); // el
int p = s.indexOf("l");       // 2
boolean eq = s.equals("HELLO"); // false
String up = s.toUpperCase();  // HELLO
```

Массивы: основное

- Длина массива задаётся при создании и дальше не меняется.
- Индексы элементов: от 0 до `length - 1`.
- `length` — поле массива, пишется без скобок.

```
int[] a = new int[5];
int[] b = {1, 3, 5};
int[] c = new int[]{1, 2, 3, 4};

a[0] = 10;

for (int i = 0; i < a.length; i++) {
    a[i] = i * i;
}

for (int x : b) {
    System.out.println(x);
}
```

Функции: общий вид

Примечание: здесь не разбираем служебные слова класса, такие как `public` и `static`. Важны только параметры, возвращаемое значение и вызов.

```
public static int sum(int a, int b) {  
    return a + b;  
}
```

Вызов:

```
int r = sum(2, 3); // 5
```

- `a`, `b` — формальные параметры;
- `2`, `3` — фактические аргументы;
- тип результата совпадает с типом возвращаемого значения.

Функции: возврат значения и void

```
int max(int a, int b) {  
    if (a > b) {  
        return a;  
    }  
    return b;  
}  
  
void printSum(int a, int b) {  
    System.out.println(a + b);  
}
```

- int — функция возвращает значение;
- void — функция не возвращает значение;
- результат функции можно использовать в выражениях:

```
int m = max(4, 7);
```

Рекурсия

Рекурсия — это функция, которая вызывает саму себя.

```
int fact(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return n * fact(n - 1);  
}  
  
int r = fact(5); // 120
```

- обязательно нужен базовый случай, например, `if (n <= 1) return;`
- без базового случая рекурсия будет бесконечной;
- каждый вызов имеет свои параметры и локальные переменные.

Насколько хороша программа?

1. Скорость выполнения

```
// лучше повторить N_TIMES раз и усреднить результат  
long start = System.nanoTime();  
  
// исследуемый блок кода  
  
long finish = System.nanoTime();  
long ns = finish - start;
```

2. Потребление памяти

3. Пользовательский опыт

В sources/Lesson6.java рассмотрели некоторые случаи и их исправления.